WashU-2 CPU

Week 3 Report

Finite State Machines — Moore FSM Design & Implementation

Author	Ayush Pati
Roll No.	25B3901
Email	25b3901@iitb.ac.in
Institution	IIT Bombay — Electrical Engineering, Second Year
Programme	SOC 2026 — WashU-2 CPU Mentorship
Submitted	Week 3 · June 2026

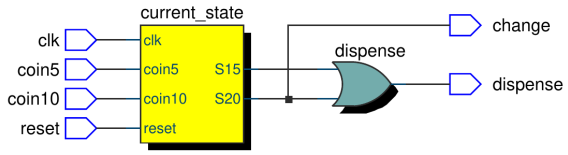
Exercise 1 — Vending Machine Controller

Entity: `vending_machine`

A Moore FSM vending machine that accepts 5p and 10p coins and dispenses a 15p item. Five states track accumulated credit: **S0** (0p), **S5** (5p), **S10** (10p), **S15** (15p — dispense), **S20** (20p — dispense + change). Inputs `coin5` and `coin10` are single-cycle pulses. Outputs `dispense` and `change` assert for exactly one clock cycle on reaching the target credit, then the FSM returns to S0. The two-process template is used: a combinational next-state/output process and a clocked state-register process.

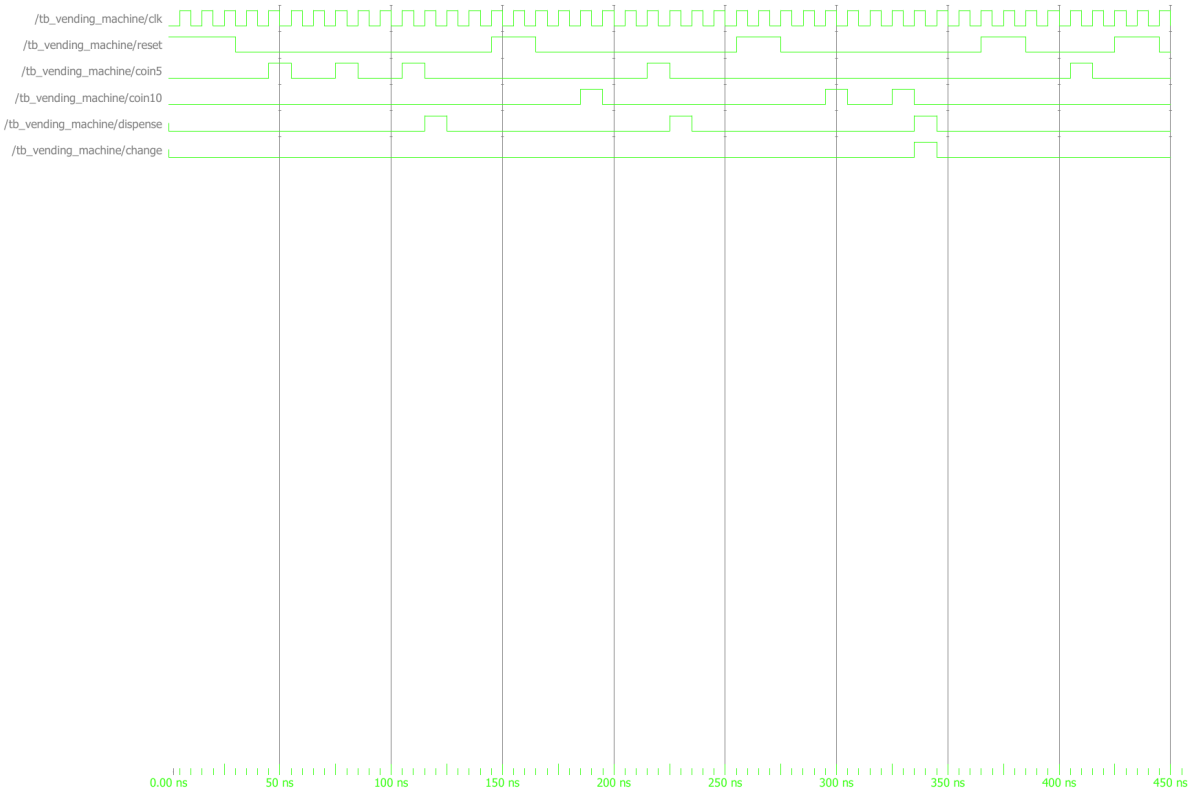
RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.



Entity:tb_vending_machine Architecture:sim Date: Thu Jun 25 17:17:56 IST 2026 Row: 1 Page: 1

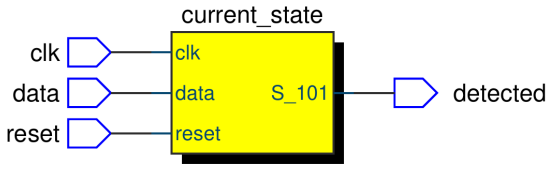
Exercise 2 — "101" Non-Overlapping Sequence Detector

Entity: `seq_det_101_nonoverlap`

A Moore FSM that detects the bit pattern "101" in a serial stream, non-overlapping: after a match the FSM restarts without reusing bits from the completed match. Four states: **S_INIT** (idle), **S_1** (seen '1'), **S_10** (seen '10'), **S_101** (match — `detected='1'` for one cycle). On leaving S_101, the incoming data bit is inspected: if `'1'`, transition to S_1 to capture the start of a possible new match; if `'0'`, return to S_INIT. This ensures correct non-overlapping behaviour while still catching back-to-back matches in streams like "101101" (two detections).

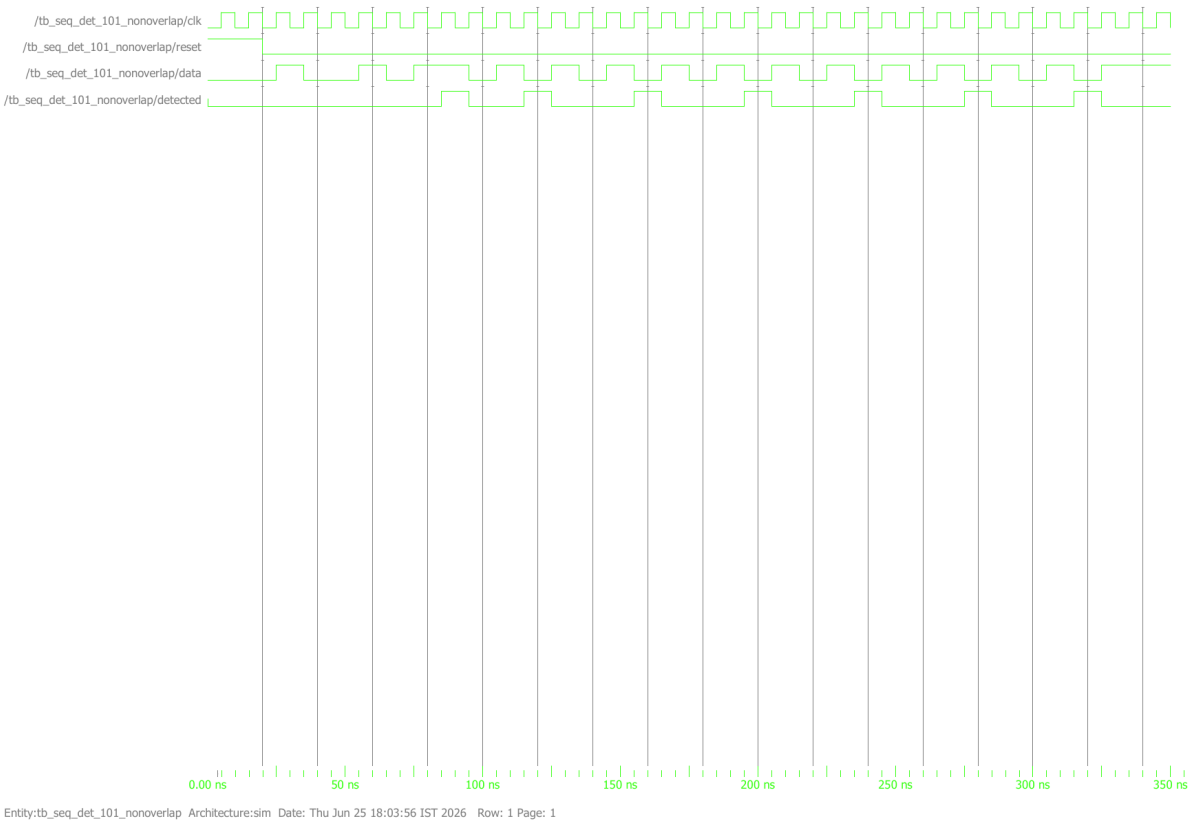
RTL Netlist View

Generated from Quartus RTL Viewer. Shows synthesised gate-level logic before technology mapping.



Simulation Waveform

Captured from ModelSim after running RTL simulation. All input and output signals are visible.


/tb_seq_det_101_nonoverlap/clk
/tb_seq_det_101_nonoverlap/reset
/tb_seq_det_101_nonoverlap/data
/tb_seq_det_101_nonoverlap/detected
0.00 ns
50 ns
100 ns
150 ns
200 ns
250 ns
300 ns
350 ns
Entity:tb_seq_det_101_nonoverlap Architecture:sim Date: Thu Jun 25 18:03:56 IST 2026 Row: 1 Page: 1

Bugs & Debugging Notes

Both bugs this week were FSM logic errors — the designs compiled and simulated without errors, but produced wrong outputs. Finding them required reading state traces carefully rather than fixing syntax.

Bug 1 — Wrong Default next_state Resets Credit on Idle Cycles (vending_machine)

The combinational process defaulted `next_state` to `S0` at the top of the case statement. The testbench inserts an idle clock cycle between each coin pulse — during that idle cycle no coin input is asserted, so the FSM saw the default and reset to S0, erasing accumulated credit before the next coin arrived.

```
-- Combinational process, top of case:
next_state <= S0; -- ■ resets every idle cycle

-- State trace: three 5p coins
-- Edge 1: coin5='1', S0 → S5
-- Edge 2: idle, S5 → S0 ← credit lost!
-- Edge 3: coin5='1', S0 → S5
-- Edge 4: idle, S5 → S0 ← credit lost again!
-- Result: machine loops S0↔S5 forever, never dispenses
```

Fixed:

```
next_state <= current_state; -- ■ hold state when no input changes it
-- Explicit S0 return inside S15/S20 cases still handles post-dispense reset

-- Correct trace: three 5p coins
-- Edge 1: coin5='1', S0 → S5
-- Edge 2: idle, S5 → S5 (held)
-- Edge 3: coin5='1', S5 → S10
-- Edge 4: idle, S10 → S10 (held)
-- Edge 5: coin5='1', S10 → S15
-- Edge 6: –, S15 → S0, dispense='1' ✓
```

In a Moore FSM two-process template, the combinational process must hold the current state when no transition condition is met — not reset to a fixed state. `next_state <= current_state` at the top of the case provides this safe default. The explicit `next_state <= S0` assignments inside the S15 and S20 cases remain correct and handle the return after dispensing — nothing else needed changing. The bug was masked in simple simulations where coins were applied back-to-back with no idle gap, and only appeared when using the provided testbench that uses `wait until rising_edge(clk)` between coin insertions.

Bug 2 — S_101 Discards Incoming Data Bit, Missing Second Match (seq_det_101_nonoverlap)

In state S_101 (match state), the combinational logic unconditionally set `next_state <= S_INIT` without inspecting the incoming `data` bit. When simulating the stream "101101", the `'1'` arriving during S_101 — the first bit of the second match — was silently discarded, causing only 1 detection instead of 2.

```
when S_101 =>
detected <= '1';
next_state <= S_INIT; -- ■ incoming data bit ignored

-- Stream '101101': only 1 detection
-- Cycle 4: data='1', in S_101 → goes to S_INIT (bit wasted)
-- Cycle 5: data='0', S_INIT → S_INIT
-- Cycle 6: data='1', S_INIT → S_1 (too late for second match)
```

Fixed:

```
when S_101 =>
detected <= '1';
if data = '1' then
next_state <= S_1; -- ■ '1' starts a new match immediately
else
next_state <= S_INIT; -- ■ '0': no useful bit, restart clean
end if;

-- Stream '101101': 2 detections ✓
-- Cycle 4: data='1', S_101 → S_1 (bit captured, detected='1')
-- Cycle 5: data='0', S_1 → S_10
-- Cycle 6: data='1', S_10 → S_101
-- Cycle 7: –, S_101 → S_INIT, detected='1' ✓
```

Non-overlapping detection means: after a match, do not reuse the closing '1' of the completed match as the start of a new one. But the *next* incoming bit (arriving while the FSM is still in S_101) is not part of the match — it is fresh data and must be processed. Checking `data` in S_101 handles this correctly: a `'1'` transitions to S_1 (a new potential match begins), while a `'0'` goes to S_INIT (no useful bit). This is still non-overlapping because when `data='0'` at S_101, the FSM returns to S_INIT rather than S_10 — it does not reuse the closing '1'. The difference between overlapping and non-overlapping is entirely in the `data='0'` branch: overlapping would go to S_10, non-overlapping goes to S_INIT.

Secondary Issue — Sync vs Async Reset Mismatch

The vending machine used a synchronous reset (checked inside `rising_edge(clk)`), but the testbench drove `reset` asynchronously mid-cycle in one phase. This created a design/testbench mismatch.

```
-- Synchronous reset (current design):
process(clk)
begin
if rising_edge(clk) then
if reset = '1' then -- only takes effect on next edge
current_state <= S0;
else
current_state <= next_state;
end if;
end if;
end process;
```

Fixed:

```
-- Asynchronous reset (matches testbench behaviour):
process(clk, reset)
begin
if reset = '1' then -- reacts immediately
current_state <= S0;
elsif rising_edge(clk) then
current_state <= next_state;
end if;
end process;
```

Asynchronous reset reacts immediately when `reset` goes high, regardless of the clock. For it to work, `reset` must be in the sensitivity list and checked before the `rising_edge` guard. Synchronous reset only takes effect on the next rising edge — safe for most testbenches that deassert reset cleanly before the next edge, but can cause a one-cycle delay if reset is toggled mid-cycle. The fix is to match the reset style between design and testbench — and to document the choice explicitly.